Manual de despliegue OpenVPN en AWS VPC y Subnet existente

Objetivo: Utilizar la VPC y la subnet pública existentes del cliente. Desde un equipo Ubuntu se conecta a GitHub, se clona el repositorio, se autentica contra la cuenta AWS, se configuran las variables de red existentes y Terraform crea la EC2 OpenVPN y los componentes administrados por la plantilla, instala OpenVPN y permite generar perfiles para usuarios.

Flujo general

- 1. Preparar Ubuntu y conectarlo con GitHub.
- 2. Clonar el repositorio terraform-aws-openvpn.
- 3. Instalar Terraform y AWS CLI.
- 4. Configurar credenciales AWS.
- 5. Identificar la VPC y subnet existentes del cliente y ajustar terraform.tfvars.
- 6. Validar que Terraform solo integre recursos sobre la red existente y desplegar.
- 7. Validar el servidor OpenVPN.
- 8. Crear y descargar perfiles .ovpn.
- 9. Actualizar el repositorio cuando se hagan cambios.

1. Prerrequisitos

- Equipo o servidor Ubuntu con acceso a Internet.
- Cuenta de GitHub con acceso al repositorio cloudtingsas/terraform-aws-openvpn.
- Cuenta AWS con permisos para crear EC2, Security Groups, Elastic IP y Key Pair.
- VPC existente, subnet pública existente y CIDR de la VPC.
- Usuario con permisos sudo en Ubuntu.

Seguridad: No almacene Access Key, Secret Key, archivos .pem, perfiles .ovpn, terraform.tfvars ni estados de Terraform dentro del repositorio Git.

1.1 Alcance

En esta modalidad la red del cliente ya existe. La plantilla se integra a esa red mediante los IDs de VPC y subnet definidos en terraform.tfvars. No se debe crear una VPC nueva para cada despliegue OpenVPN, salvo que el diseño del cliente lo requiera expresamente.

Componente	Estado	Responsable
VPC	Debe existir antes del despliegue	Cliente / infraestructura existente
Subnet pública	Debe existir antes del despliegue	Cliente / infraestructura existente
Internet Gateway y ruta a Internet	Deben permitir salida/entrada según el diseño de la subnet pública	Cliente / infraestructura existente
EC2 Ubuntu OpenVPN	Se crea con Terraform	Plantilla Terraform
Security Group OpenVPN	Se crea/configura con Terraform	Plantilla Terraform
Elastic IP	Se crea si create_eip = true	Plantilla Terraform
Key Pair / llave SSH	Se crea según la plantilla actual	Plantilla Terraform
OpenVPN y cliente inicial	Se instalan/configuran durante el aprovisionamiento	Plantilla Terraform

Punto clave: La conexión a la cuenta AWS no crea recursos por sí sola. AWS CLI entrega la autenticación; Terraform usa esas credenciales para crear los recursos declarados en el código sobre la VPC y subnet existentes.

1.2 Flujo de extremo a extremo

```
Ubuntu administrativo
  |
  +--> GitHub: clonar terraform-aws-openvpn
  |
  +--> AWS CLI: autenticar contra la cuenta del cliente
  |
  +--> terraform.tfvars: VPC + subnet existentes
  |
  +--> terraform init / plan / apply
            |
            +--> AWS VPC existente
                    |
                    +--> Subnet pública existente
                            |
                            +--> EC2 OpenVPN
```

```
                              +--> Security Group
                              +--> Elastic IP (opcional)
                              +--> Key Pair / acceso SSH
                              +--> Configuración OpenVPN
```

2. Preparar Ubuntu y conectar con GitHub

2.1 Actualizar el sistema e instalar Git

```
sudo apt update
sudo apt install -y git openssh-client
git --version
ssh -V
```

2.2 Configurar identidad de Git

```
git config --global user.name "Nombre Apellido"
git config --global user.email "correo@empresa.com"
git config --global --list
```

2.3 Crear una llave SSH para GitHub

```
ssh-keygen -t ed25519 -C "correo@empresa.com"
```

Presione Enter para usar la ruta sugerida. Se recomienda proteger la llave privada con una passphrase cuando el equipo sea de uso administrativo.

```
eval "$(ssh-agent -s)"
ssh-add ~/.ssh/id_ed25519
cat ~/.ssh/id_ed25519.pub
```

Copie el contenido mostrado y agréguelo en GitHub en Settings > SSH and GPG keys > New SSH key.

2.4 Probar la conexión con GitHub

```
ssh -T git@github.com
```

La primera vez puede solicitar confirmar la huella del host. Si la autenticación es correcta, GitHub confirmará que la llave fue aceptada.

Alternativa: También puede clonar por HTTPS, pero para repositorios privados GitHub requiere autenticación con un token o un método autorizado. Para servidores Ubuntu administrativos, SSH suele ser más cómodo para operaciones repetidas.

3. Clonar el repositorio OpenVPN

Cree una carpeta de trabajo y clone el repositorio:

```
mkdir -p ~/proyectos
cd ~/proyectos
git clone git@github.com:cloudtingsas/terraform-aws-openvpn.git
cd terraform-aws-openvpn
git status
ls -la
```

La estructura esperada debe incluir, como mínimo:

```
main.tf
variables.tf
outputs.tf
terraform.tfvars.example
README.md
scripts/crear-cliente-openvpn.sh
docs/
```

Importante: Si el script crear-cliente-openvpn.sh todavía no está integrado dentro del aprovisionamiento de Terraform, el manual incluye más adelante cómo copiarlo manualmente a la instancia EC2.

4. Instalar Terraform en Ubuntu

Instale Terraform desde el repositorio oficial de HashiCorp:

```
sudo apt-get update && sudo apt-get install -y gnupg software-properties-common curl
curl -fsSL https://apt.releases.hashicorp.com/gpg | sudo gpg --dearmor -o
/usr/share/keyrings/hashicorp-archive-keyring.gpg
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(. /etc/os-release && echo $VERSION_CODENAME) main" |
sudo tee /etc/apt/sources.list.d/hashicorp.list
sudo apt update
sudo apt install -y terraform
terraform version
```

5. Instalar AWS CLI

Para Ubuntu x86_64:

```
sudo apt update
sudo apt install -y unzip curl
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
unzip awscliv2.zip
sudo ./aws/install
aws --version
```

Si el servidor Ubuntu es ARM64, debe utilizar el instalador correspondiente a aarch64 en lugar de x86_64.

6. Configurar credenciales AWS

No escriba credenciales dentro de main.tf. Configure AWS CLI:

```
aws configure --profile openvpn
```

Ingrese AWS Access Key ID, AWS Secret Access Key, región y formato de salida. Luego active el perfil:

```
export AWS_PROFILE=openvpn
aws sts get-caller-identity
```

Validación: No continúe con Terraform hasta que aws sts get-caller-identity muestre correctamente la cuenta y el principal IAM con el que se realizará el despliegue.

7. Configurar el proyecto para el cliente

Antes de ejecutar Terraform, obtenga de la cuenta AWS del cliente la región, el ID de la VPC, el CIDR de esa VPC y el ID de una subnet pública existente. La subnet debe tener la conectividad requerida hacia Internet para que la EC2 OpenVPN pueda operar según el diseño actual.

7.1 Identificar la red existente

```
aws ec2 describe-vpcs --region us-east-1 --query "Vpcs[*].[VpcId,CidrBlock,Tags]" --output
table
aws ec2 describe-subnets --region us-east-1 --filters
Name=vpc-id,Values=vpc-xxxxxxxxxxxxxxxxx --query
"Subnets[*].[SubnetId,CidrBlock,AvailabilityZone,MapPublicIpOnLaunch,Tags]" --output table
```

Validación de red: Confirme con el responsable de red cuál subnet pública debe utilizarse. No seleccione una subnet solo por el nombre; valide sus rutas y controles de seguridad.

7.2 Crear terraform.tfvars

```
cp terraform.tfvars.example terraform.tfvars
nano terraform.tfvars
```

Ejemplo de variables, manteniendo la estructura del proyecto actual:

```
aws_region = "us-east-1"
vpc_id = "vpc-xxxxxxxxxxxxxxxxx"
vpc_cidr = "10.60.0.0/16"
subnet_id = "subnet-xxxxxxxxxxxxxxxxx"
name_prefix = "BastionOpenVPN"
key_name = "BastionOpenVPN"
instance_type = "t3.micro"
openvpn_allowed_cidrs = ["203.0.113.10/32"]
initial_client_name = "cliente1"
create_eip = true
```

7.3 Significado de las variables

Variable	Uso
aws_region	Región AWS donde se desplegará OpenVPN.
vpc_id	VPC donde se creará la instancia EC2.
vpc_cidr	Red de la VPC que será accesible a través de la VPN.
subnet_id	Subnet pública donde se ubicará el servidor OpenVPN.
name_prefix	Prefijo para los recursos creados.
key_name	Nombre de la llave SSH generada para administrar la instancia.
instance_type	Tipo de instancia EC2.
openvpn_allowed_cidrs	CIDR públicos autorizados para conectarse al puerto OpenVPN.
initial_client_name	Nombre del primer perfil OpenVPN.
create_eip	Define si se asigna Elastic IP al servidor.

8. Desplegar OpenVPN con Terraform sobre la red existente

Desde la raíz del repositorio:

```
cd ~/proyectos/terraform-aws-openvpn
```

8.1 Inicializar

```
terraform init
```

8.2 Formatear y validar

```
terraform fmt -recursive
terraform validate
```

8.3 Revisar el plan

```
terraform plan
```

Revise especialmente región, VPC y subnet. Confirme que corresponden a la infraestructura existente del cliente. En el plan deben aparecer como recursos a crear la EC2 y los componentes definidos por la plantilla; la VPC y la subnet no deberían aparecer como nuevas.

No continuar si: terraform plan muestra la creación o reemplazo de una VPC/subnet que el cliente ya utiliza, o si muestra cambios destructivos sobre infraestructura no asociada al OpenVPN. Revise main.tf, variables y terraform.tfvars antes del apply.

8.4 Aplicar

```
terraform apply
```

Cuando Terraform solicite confirmación, escriba yes. Al finalizar, consulte los outputs:

```
terraform output
terraform output ssh_command
terraform output download_client_ovpn_command
```

9. Acceder a la instancia OpenVPN

Si Terraform generó localmente la llave .pem, restrinja sus permisos:

```
chmod 400 BastionOpenVPN.pem
```

Obtenga el comando SSH desde Terraform o utilice la IP pública:

```
ssh -i BastionOpenVPN.pem ubuntu@IP_PUBLICA
```

10. Validar que OpenVPN está funcionando

```
sudo systemctl status openvpn-server@server
sudo ss -lunp | grep 1194
sudo cat /etc/openvpn/server/server.conf
ls -lh ~/client-configs/files/
```

Resultado esperado: El servicio debe estar activo, el servidor debe escuchar en UDP 1194 y debe existir al menos el perfil inicial configurado durante el despliegue.

Seguridad: Cada usuario debe tener un perfil .ovpn independiente. No publique archivos .ovpn ni llaves .pem en GitHub, correo abierto, chats públicos o repositorios compartidos.

11. Creación y administración básica de usuarios OpenVPN

Una vez finalizado terraform apply y validado que el servicio OpenVPN está activo, los perfiles de usuario se crean desde la instancia EC2 mediante el script crear-cliente-openvpn.sh. El procedimiento siguiente complementa el despliegue de infraestructura y deja el flujo operativo de usuarios alineado con el script actual.

11.1 Validar el servicio OpenVPN

Conéctese a la EC2 OpenVPN y confirme que el servicio y el puerto UDP 1194 están disponibles:

```
ssh -i BastionOpenVPN.pem ubuntu@IP_PUBLICA
sudo systemctl status openvpn-server@server
sudo ss -lunp | grep 1194
```

El estado esperado del servicio es Active: active (running).

11.2 Validar que el script exista en la EC2

El script debe quedar disponible en /home/ubuntu/crear-cliente-openvpn.sh. Valídelo con:

```
ls -lh /home/ubuntu/crear-cliente-openvpn.sh
```

Si no tiene permisos de ejecución:

```
chmod +x /home/ubuntu/crear-cliente-openvpn.sh
```

Se recomienda que el archivo pertenezca al usuario ubuntu:

```
sudo chown ubuntu:ubuntu /home/ubuntu/crear-cliente-openvpn.sh
```

Si Terraform todavía no copia automáticamente el script a la EC2, desde el equipo Ubuntu administrativo donde clonó GitHub ejecute:

```
scp -i BastionOpenVPN.pem scripts/crear-cliente-openvpn.sh
ubuntu@IP_PUBLICA:/home/ubuntu/crear-cliente-openvpn.sh
ssh -i BastionOpenVPN.pem ubuntu@IP_PUBLICA
chmod +x ~/crear-cliente-openvpn.sh
```

11.3 Prerrequisitos del script

Antes de crear un usuario, el servidor debe tener disponibles los componentes que utiliza el script:

- /etc/openvpn/easy-rsa
- /etc/openvpn/server/ca.crt
- /etc/openvpn/server/tls-crypt.key
- /home/ubuntu/client-configs/files

El script obtiene automáticamente el DNS público o la IP pública de la EC2 mediante AWS Instance Metadata. Por este motivo la instancia debe disponer de una IP pública o Elastic IP asociada en el diseño actual.

11.4 Crear un usuario OpenVPN

Ejecute el script desde la sesión del usuario ubuntu. No lo ejecute con sudo.

Sintaxis:

```
~/crear-cliente-openvpn.sh nombre_usuario
```

Ejemplo:

```
~/crear-cliente-openvpn.sh carlos.rojas
```

El nombre puede contener letras, números, punto, guion y guion bajo. El script rechaza otros caracteres.

Importante: la firma del certificado se realiza automáticamente en modo batch; no es necesario ejecutar manualmente Easy-RSA ni confirmar la firma del certificado.

11.5 Qué realiza automáticamente el script

- Impide la ejecución como root/sudo para evitar generar los perfiles en un contexto incorrecto.
- Valida el nombre recibido para el cliente.
- Obtiene el DNS público o la IP pública de la EC2 desde AWS Metadata.
- Valida Easy-RSA, la CA del servidor y la llave tls-crypt.
- Genera la solicitud y llave privada del cliente con Easy-RSA.
- Firma el certificado como cliente.
- Construye un único archivo .ovpn con la configuración de conexión, CA, certificado, llave privada y tls-crypt.
- Guarda el perfil en /home/ubuntu/client-configs/files.
- Aplica permisos 600 al archivo .ovpn y lo deja como propiedad de ubuntu.

11.6 Resultado esperado

Al terminar correctamente debe mostrarse un mensaje indicando el archivo generado y el servidor remoto utilizado. Para el ejemplo anterior, el perfil será:

```
/home/ubuntu/client-configs/files/carlos.rojas.ovpn
```

Valide su existencia y permisos:

```
ls -lh ~/client-configs/files/carlos.rojas.ovpn
```

El resultado debe mostrar un archivo propiedad de ubuntu y con permisos restrictivos equivalentes a -rw-------.

11.7 Validar el contenido básico del perfil

Sin exponer el contenido completo de la llave privada, puede verificar que el perfil incluya las secciones requeridas:

```
grep -E "client|remote|<ca>|<cert>|<key>|<tls-crypt>" ~/client-configs/files/carlos.rojas.ovpn
```

Debe observar al menos client, una directiva remote con UDP 1194 y las secciones <ca>, <cert>, <key> y <tls-crypt>.

11.8 Si el usuario ya existe

Si el certificado y la llave privada del cliente ya existen, el script no vuelve a emitirlos. Regenera únicamente el archivo .ovpn utilizando el material existente. Esto puede utilizarse cuando se eliminó el perfil local o cuando se necesita volver a construirlo con el DNS/IP pública actual.

```
~/crear-cliente-openvpn.sh carlos.rojas
```

11.9 Crear usuarios adicionales

Repita el mismo comando cambiando el nombre. Cada persona debe tener su propio certificado y perfil .ovpn:

```
~/crear-cliente-openvpn.sh maria.gomez
~/crear-cliente-openvpn.sh soporte1
~/crear-cliente-openvpn.sh proveedor-vpn
```

Para listar los perfiles generados:

```
ls -lh ~/client-configs/files/
```

12. Descargar, importar y validar el perfil .ovpn

12.1 Descargar el perfil desde el equipo administrador

Ejecute SCP desde el equipo Ubuntu administrativo, no desde la EC2 OpenVPN:

```
scp -i BastionOpenVPN.pem ubuntu@IP_PUBLICA:~/client-configs/files/carlos.rojas.ovpn .
```

También puede utilizar el DNS público mostrado por el script en lugar de IP_PUBLICA.

12.2 Entregar e importar el perfil

El archivo .ovpn contiene la llave privada del usuario y debe entregarse únicamente por un canal seguro al usuario correspondiente. No reutilice un mismo perfil entre varias personas.

En el equipo del usuario:

- Abrir OpenVPN Connect.
- Seleccionar la opción para importar un perfil.
- Seleccionar el archivo .ovpn entregado.
- Guardar el perfil y conectar.
- Confirmar que el cliente quede en estado conectado.

12.3 Validar la conexión desde el servidor

Después de que el usuario se conecte, revise el registro de estado configurado por OpenVPN:

```
sudo cat /var/log/openvpn/status.log
```

Para monitorear en tiempo real:

```
sudo tail -f /var/log/openvpn/status.log
```

Cuando el cliente esté conectado debe aparecer una entrada correspondiente al certificado del usuario y una dirección del rango VPN configurado, por ejemplo 10.8.0.x si ese es el pool utilizado por el servidor.

12.4 Seguridad de los perfiles

- Un perfil .ovpn por cada usuario.
- No subir archivos .ovpn al repositorio GitHub.
- No enviar perfiles por canales públicos o grupos compartidos.
- Mantener un inventario de usuarios y perfiles entregados.
- Si se sospecha exposición de un perfil, el certificado debe revocarse antes de generar un reemplazo.
- La revocación de certificados no está automatizada por el script crear-cliente-openvpn.sh actual y debe manejarse como procedimiento separado.

13. Actualizar el repositorio en Ubuntu

Cuando el repositorio cambie en GitHub, actualice la copia local antes de trabajar:

```
cd ~/proyectos/terraform-aws-openvpn
git status
git pull origin main
```

Si usted modifica documentación, scripts o Terraform desde Ubuntu:

```
git status
git add .
git commit -m "chore: actualizar plantilla OpenVPN"
git pull origin main --rebase
git push origin main
```

Antes del push: Revise siempre git status y confirme que no aparezcan terraform.tfvars, .pem, .ovpn, .tfstate o credenciales.

14. .gitignore recomendado

```
.terraform/
*.tfstate
*.tfstate.*
terraform.tfvars
*.auto.tfvars
*.auto.tfvars.json
*.pem
*.key
*.ovpn
.env
.env.*
.DS_Store
```

15. Buenas prácticas de seguridad

- Restringir SSH/22 únicamente a IPs administrativas o a una red de gestión.
- Evitar 0.0.0.0/0 en openvpn_allowed_cidrs salvo que exista una justificación operativa. Preferir /32 de las IP públicas autorizadas cuando sean estables.
- Permitir únicamente UDP 1194 para OpenVPN, salvo que la configuración del proyecto utilice otro puerto/protocolo.
- No reutilizar el mismo .ovpn entre varios usuarios.

- Mantener inventario de usuarios creados y revocar perfiles cuando un usuario deje de requerir acceso.
- Guardar el estado Terraform en un backend seguro si el proyecto pasa de una administración individual a un proceso de equipo/producción.
- Revisar terraform plan antes de cada terraform apply.

16. Eliminación de la infraestructura

Si se requiere eliminar los recursos creados por esta plantilla:

```
terraform destroy
```

Revise el plan de destrucción y confirme escribiendo yes solo cuando esté seguro.

17. Checklist de despliegue

- Ubuntu actualizado y Git instalado.
- Llave SSH registrada en GitHub y ssh -T git@github.com validado.
- Repositorio clonado correctamente.
- Terraform y AWS CLI instalados.
- aws sts get-caller-identity validado.
- VPC existente, CIDR y subnet pública validados con el cliente.
- terraform.tfvars creado con los IDs de la red existente.
- terraform validate sin errores.
- terraform plan revisado y confirmado que no crea/reemplaza la VPC o subnet existente.
- terraform apply completado.
- Servicio openvpn-server@server activo.
- UDP 1194 escuchando.
- Perfil inicial .ovpn generado y probado.
- Proceso de creación de usuarios validado.
- Archivos sensibles fuera de GitHub.